

Поиск релевантных статей поддержки Avito

Решение возвращает десять статей справочного центра для каждого обращения. Весь код обработки данных, исследования, валидации, обучения и подготовки `answer.csv` находится в `solution.ipynb`. Для обычного запуска приложены совместимые artifacts, поэтому повторять наиболее долгие нейросетевые расчеты не требуется.

Контекст задачи

В наборе находятся 793 статьи, 500 размеченных запросов в `calibration.f` и 500 запросов в `test.f`. Одному запросу может соответствовать несколько статей. Многие статьи встречаются в разметке редко, поэтому решение сочетает текстовый поиск, реранк и признаки похожих обращений.

Обработка HTML

Из HTML извлекаются заголовки, абзацы, списки, таблицы, вкладки, спойлеры и внутренние ссылки. Для каждой статьи формируются полный очищенный текст и короткие смысловые фрагменты. Полный текст нужен для определения общей темы, а фрагменты помогают найти конкретную инструкцию внутри длинной страницы.

Поиск и ранжирование

Первые 50 кандидатов собираются по словному и символьному TF-IDF, BM25, TF-IDF по фрагментам, эмбедингам Qwen3-Embedding-0.6B, плотным и разреженным представлениям BGE-M3, а также по похожим размеченным обращениям. BGE-reranker-v2-m3 оценивает запрос вместе с двумя наиболее подходящими фрагментами статьи.

Для каждой пары запрос-статья рассчитываются позиции и оценки всех поисковых методов, согласие между ними, близость к размеченным запросам, прототипы формулировок и связи между статьями. `LogisticRegression` упорядочивает кандидатов по этим признакам. Ее список объединяется со структурным ранжированием при помощи RRF с `k=30` и равными весами.

Проверка качества

Основная проверка использует пять мультитейл-стратифицированных фолдов с `seed=42`. Все признаки, зависящие от разметки, строятся только по обучающей части текущего фолда. Параметры выбираются на внутреннем сплите, после чего MAP@10 рассчитывается по объединенным OOF-предсказаниям всех 500 запросов.

Правильные статьи почти всегда попадают в первые 50 кандидатов, поэтому основная проблема находится не в поиске, а в порядке результатов. Сначала мы усилили реранк короткими фрагментами статьи. Затем добавили прототипы запросов, тексты внутренних ссылок и связи между статьями, чтобы лучше различать близкие инструкции. После этого проверили линейное обучение, бустинг, другой реранкер и попарное обучение, ориентированное на перестановки внутри top-10.

Группа	Вариант	OOF MAP@10	Итог
Поиск и реранк	BGE/Qwen и короткие фрагменты	0.712290	основа следующих опытов
Структурные признаки	полный набор	0.717725	вошли в линейный RRF
Линейная модель	LogisticRegression	0.720497	улучшила порядок кандидатов
Линейная модель	LogisticRegression и RRF	0.726328	выбранная подтвержденная схема
Линейная модель	исправленный pointwise Linear/RRF	0.724767	текущая контрольная проверка
Линейная модель	AP@10-взвешенный pairwise Linear/RRF	0.721341	хуже основной метрики
Порог уверенности	условное добавление статей	0.728105	историческая схема валидации
LightGBM	те же признаки	0.706230	хуже линейной модели
BGE-M3	самостоятельное ранжирование	0.433207	хуже ансамбля сигналов
MiniLM	сложные отрицательные примеры	0.621153	хуже базового реранка
XGBRanker	rank:pairwise	0.738757	историческая схема валидации
XGBRanker	rank:ndcg	0.757047	историческая схема валидации
RRF	объединение трех моделей	0.737547	историческая схема валидации

Тут +- все итерации отразил решения, за исключением может быть самых первых, который давали метрику меньше 0.5.

Ошибки

Для каждого OOF-запроса рассчитываются AP@10, полнота кандидатов и позиции всех правильных статей. Чаще всего модель ошибается на первом месте или возвращает неполный набор для запроса с несколькими ответами. Линейное ранжирование уменьшает такие ошибки за счет согласия поисковых методов и похожих размеченных обращений. Для редких статей важнее признаки текста и ссылок, поскольку частота метки почти не помогает. Самые характерные примеры выбираются программно из OOF, без заранее заданных `query_id`.

Запуск

Рекомендуется Python 3.11 и GPU с 8 ГБ видеопамяти. PyTorch устанавливается отдельно, чтобы выбрать подходящую CUDA или CPU сборку. Остальные зависимости зафиксированы в `requirements.txt`.

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install torch==2.10.0+cu128 --index-url
https://download.pytorch.org/whl/cu128
pip install -r requirements.txt
jupyter nbconvert --to notebook --execute --inplace `
--ExecutePreprocessor.timeout=7200 solution.ipynb
```

Обычный запуск использует данные из `artifacts/`, проверяет их совместимость и быстро формирует финальный файл. Если artifacts отсутствуют или установлено `FORCE_RECOMPUTE=True`, notebook может заново пройти полный путь от HTML до ответа, но может занять много времени.

Модель	Commit
Qwen3-Embedding-0.6B	97b0c614be4d77ee51c0cef4e5f07c00f9eb65b3
BGE-M3	c01c5b303a87da9c91d581256f0af47b5e0a9bd1
BGE-reranker-v2-m3	953dc6f6f85a1b2dbfca4c34a2796e7dde08d41e

Состав

- `solution.ipynb`: исследование и весь исполняемый код;
- `artifacts/`: совместимые кэши выбранной конфигурации и совместных экспериментов;
- `candidate_data/`: исходные данные;
- `requirements.txt`: зависимости, кроме PyTorch;
- `answer.csv`: итоговый файл ответа.